

8번 블록암호

중간 상태 누출을 이용한 변형 AES 마스터 키 복원

암호분석경진대회 제출 보고서

2026년 7월

1 결과 요약

복원한 16바이트 마스터 키는 다음과 같다.

2923be84e16cd6ae529049f1f1bbe9eb

문제의 0-based index 28178에 해당하는 평문 · 암호문과 leaked.txt를 결합했다. 첫 · 마지막 라운드 식에서 $K[0]=29$, $K[1]=23$, $K[3]=84$ 를 직접 구하고, X^3 의 2^{16} 개 완성과 X^2 첫 열의 16개 완성을 공유 마스터 키 바이트로 결합했다. 중복을 제거한 full key 중 누출 대상 암호문을 만족하는 키는 정확히 하나였으며, 그 키로 제공된 50,000개 평문을 모두 다시 암호화한 결과 불일치는 0개였다.

검증 항목	결과
누출 대상 index	28178 (0부터 시작)
직접 복원한 마스터 키 바이트	$K[0]$, $K[1]$, $K[3]$
누출 대상 pair를 만족하는 서로 다른 full key	1개
전체 재암호화	50,000개 중 mismatch 0개

2 문제 암호와 상태 표현

AES state는 표준의 column-major 순서를 사용한다.

$$index = row + 4 \cdot column.$$

SubBytes, ShiftRows, MixColumns를 합쳐

$$F = MC \circ SR \circ SB$$

라 쓰면 문제의 변형 7라운드 암호는

$$X^0 = P, \quad X^{i+1} = F(X^i) \oplus k_i \quad (0 \leq i < 7), \quad C = X^7$$

이다. 표준 AES와 달리 초기 AddRoundKey가 없고 마지막 라운드에도 MixColumns가 있다. 또한 라운드 키는 표준 AES key expansion이 아니라, 마스터 키 바이트의 문제 지정 permutation에 일부 라운드의 MixColumns를 적용해 만든다. 특히 $k_0 = K$ 이며, 서로 떨어진 라운드가 같은 마스터 키 바이트를 공유한다. 이 공유 관계가 여러 중간 상태 누출을 작은 key-byte 제약으로 연결하게 해 준다.

2.1 힌트와 실제 공격 범위

문제는 Demirci-Selçuk meet-in-the-middle 공격을 힌트로 준다. 원 논문은 5라운드 distinguisher를 바탕으로 reduced-round AES를 공격한다. 그러나 본 solver는 그 distinguisher, 논문의 다라운드 backward 경로, 또는 논문에 제시된 AES-192/256 공격 복잡도를 구현하지 않는다.

여기서는 문제에 훨씬 강한 중간 상태 bit 누출과 특수한 라운드 키 permutation이 직접 주어진다. 원 공격에서 참고한 것은 중간 표현의 후보를 미리 계산하고 공통 값을 기준으로 결합한다는 관점뿐이다. 따라서 실제로 구현한 방법은 **leak-assisted bidirectional key-byte constraint join**으로 구분한다. 이를 고전 Demirci-Selçuk 공격의 구현이나 성능 재현이라고 주장하지 않는다.

3 누출 대상과 직접 복원

3.1 0-based index 검증

배열의 28178번째 block을 읽은 결과는 다음과 같다.

```
P[28178] = b3a6db3c870c3e99245e0d1c06b747de
C[28178] = 0b764cbfd5a347a13b41b4fdd5b11e00
```

두 값은 leaked.txt의 X^0 , X^7 전체 byte와 일치한다. 따라서 자연어의 “28,179번째”를 1-based 배열 첨자로 오해하지 않고 문제에 명시된 0-based index를 사용했다.

3.2 첫 라운드의 $K[0]$

$k_0 = K$ 이고 $X^1 = F(X^0) \oplus k_0$ 이다. 누출에는 $X^1[0] = 20$ 이 완전히 주어지므로

$$K[0] = F(X^0)[0] \oplus X^1[0] = 29$$

를 바로 얻는다.

3.3 마지막 라운드의 $K[1]$, $K[3]$

마지막 라운드 키는

$$k_6 = \text{MC}(K[1], K[5], K[7], K[14], \dots, K[9], K[13], K[3], K[6], \dots)$$

형태다. $X^7 = F(X^6) \oplus k_6$ 의 각 열에 InvMixColumns를 적용하면 MixColumns의 선형성 때문에 다음 관계가 된다.

$$\text{InvMC}(C_{\text{col}0}) \oplus [K_1, K_5, K_7, K_{14}] = \text{SR}(\text{SB}(X^6))_{\text{col}0},$$

$$\text{InvMC}(C_{\text{col}2}) \oplus [K_9, K_{13}, K_3, K_6] = \text{SR}(\text{SB}(X^6))_{\text{col}2}.$$

$X^6[0]$ 과 $X^6[2]$ 가 완전히 누출되어 있으므로

$$K[1] = 23, \quad K[3] = 84$$

를 직접 복원한다. 이로써 탐색 전에 마스터 키 3바이트가 확정된다.

4 누출 completion과 후보 결합

최종 구현에서 사용하는 누출은 다음과 같다.

- $X^1[0]$: 완전한 1바이트;
- $X^2[0..3]$: 각 바이트에 unknown bit 1개;
- $X^3[0..15]$: 각 바이트에 unknown bit 1개;
- $X^4[0], X^4[5], X^4[10], X^4[15]$: 각 바이트에 unknown bit 1개;
- $X^6[0], X^6[2]$: 완전한 2바이트; 그리고
- X^0, X^7 : 완전한 평문과 암호문.

4.1 $X^3 \rightarrow X^4$ 대각선

X^3 의 각 바이트에는 unknown bit가 하나이므로 가능한 전체 state는 $2^{16} = 65,536$ 개다. 각 completion마다 $F(X^3)$ 와 필요한 네 InvMixColumns 열을 미리 계산해 record로 보관한다. X^4 의 대각 위치 0, 5, 10, 15와 k_3 의 단순 마스터 키 permutation을 대조하면 다음 후보 집합을 얻는다.

마스터 키 바이트	서로 다른 후보 수
K[10]	32
K[11]	16
K[12]	32
K[13]	32

4.2 X^2 첫 열과 compact relation

$X^2[0..3]$ 에도 각각 unknown bit가 하나뿐이므로 첫 열 completion은 $2^4 = 16$ 개다. MixColumns의 선형성을 이용하면

$$\text{InvMC}(X_{\text{col}0}^2) = \text{SB}(X_{\text{diag}}^1) \oplus [K_5, K_8, K_6, K_3]$$

의 형태로 바꿀 수 있다. 이미 확정된 $K[3]$ 과 $K[10]$ 후보 32개를 결합하면

$$16 \cdot 32 = 512$$

개의 기본 X^2 /key relation만 남는다. 이 단계에서 서로 다른 후보 수는 $K[5]$, $K[8]$, $K[15]$ 가 각각 16개이고, $K[6]$ 은 192개다.

4.3 실제 leak-assisted join

solver의 결합 순서는 다음과 같다.

1. 앞뒤 직접식으로 $K[0]$, $K[1]$, $K[3]$ 을 확정한다.
2. X^3 의 2^{16} 개 completion을 record로 만든다.
3. 각 record의 $F(X^3)$ 대각 byte와 X^4 대각 누출을 결합해 $K[10..13]$ 후보를 만든다.
4. X^2 의 16개 completion과 X^3 record를 공유 key byte로 비교한다. 이 직접 비교 측은 $16 \cdot 2^{16} = 2^{20} = 1,048,576$ 개다.
5. 살아남은 partial key에서 $K[2]$, $K[4]$, $K[14]$ 를 전방 · 역방향 X^2 가 완전히 같다는 조건으로 채운다.

최종 코드는 명시적인 X^5 역계산이나 $X^7 \rightarrow X^4$ backward table을 만들지 않는다. 여러 라운드 키에 재사용된 동일 마스터 키 바이트가 곧 join key이며, 누출이 그 값들을 빠르게 고정한다.

5 마지막 3바이트의 완전 탐색과 유일성

`complete_partial_keys()`는 한 partial key에서 처음 발견한 조합을 반환하지 않는다. 먼저 $K[2]$ 와 $K[4]$ 를 각각 0부터 255까지 별도 목표 byte로 거른 뒤, 살아남은 모든 조합에 대해 $K[14]$ 의 256개 값을 끝까지 조사한다. 후보마다

$$X_{\text{forward}}^2(K) = F(X^1) \oplus k_1$$

와

$$X_{\text{backward}}^2(K) = F^{-1}(X^3 \oplus k_2)$$

의 16바이트 전체가 같은지 확인한다.

생성된 모든 full key는 `set[bytes]`에 넣어 서로 다른 경로가 같은 키를 만드는 경우를 중복 제거한다. 그 뒤 누출 대상 평문을 직접 암호화해 해당 암호문과 일치하는 키만 남긴다. 서로 다른 생존 key가 정확히 하나인지 검사한 뒤에만 결과를 반환하므로, “첫 해를 정답으로 간주”하는 조기 종료에 의존하지 않는다.

6 복잡도와 측정 범위

단계	후보 또는 작업 수
X^3 의 16개 unknown bit completion	$2^{16} = 65,536$
$X^2[0..3]$ 의 4개 unknown bit completion	$2^4 = 16$
기본 $X^2/K[10]$ relation	$16 \cdot 32 = 512$
X^2 completion과 X^3 record 비교	$2^{20} = 1,048,576$
최종 사후 검증	50,000 blocks · 7 rounds

메모리의 주된 항은 65,536개 `x3_records`다. 현재 Python object 표현에서는 공간 복잡도가 $O(2^{16})$ 이며, packed integer나 고정폭 array를 쓰면 상수 계수는 줄일 수 있다.

Python 3.11.2, Linux 6.1.0-51-cloud-amd64, glibc 2.36 환경의 같은 process에서 key 탐색, 완전 completion, 50,000쌍 검증을 함께 켜 두 실행은 41.706–57.160초, peak RSS 59,812–60,436 KiB (약 58.4–59.0 MiB)였다. 이는 통제된 다회 benchmark가 아니며 Python version, allocator, CPU 부하에 따라 달라진다. 따라서 위 후보 수는 공격 구조를 설명하는 값이고, wall time과 RSS는 현재 구현의 제한된 end-to-end 관측값일 뿐 복잡도의 증명이나 다른 장비의 성능 보장이 아니다.

6.1 SMT probe의 비결론성

탐색 초기에 임시 z3-solver 환경에서 full AES bit-vector 모델과 state/key layout 변형을 짧게 시도했으나 timeout에 해당하는 unknown으로 끝났다. 당시 probe code와 정확한 timeout 기록을 보존하지 않았으므로 이는 재현 가능한 실패 실험이 아니다. 특히 unknown은 UNSAT나 SMT 접근의 불가능성을 뜻하지 않는다. 최종 방법을 선택한 근거는 leak-assisted join이 실제 key를 복원하고 전체 50,000쌍으로 검증됐다는 양성 결과다.

7 구현, 재현 및 검증

최종 Python solver는 AES 정 · 역연산, 문제의 라운드 키 생성, leak parser, 후보 record와 join을 구현한다. 마지막 세 key byte를 빠짐없이 생성하고 전체 데이터를 검증하는 과정도 Python 표준 라이브러리만 사용한다. 저장소 루트에서 다음을 실행한다.

```
python3 solutions/investigate_08_aes_leak.py
python3 solutions/solve_08_aes_key.py
```

최종 solver의 핵심 출력은 다음과 같다.

```
master key = 2923be84e16cd6ae529049f1f1bbe9eb
verified pairs = 50000
mismatches = 0
submission key check = PASS
```

실행 중에는 다음도 함께 검사한다.

- 누출 index의 X^0 , X^7 과 제공 파일의 평문 · 암호문 일치;
- 직접 복원한 3바이트와 누출 bit pattern의 일치;
- 모든 completion에서 생성한 full key의 중복 제거와 유일성;
- 복원 key로 50,000개 block 전체를 재암호화한 mismatch 0; 그리고
- 출력 key와 submissions/08/master_key.txt의 byte-for-byte 일치.

8 참고 자료

- [NIST, FIPS 197: Advanced Encryption Standard \(AES\), 2023 update](#). Column-major state, SubBytes, ShiftRows, MixColumns와 역변환 구현의 기준으로 사용했다.
- [Hüseyin Demirci and Ali Aydın Selçuk, A Meet-in-the-Middle Attack on 8-Round AES, FSE 2008](#). 중간 표현 후보의 precompute/join 관점을 참고했으며, 본 solver가 논문의 distinguisher와 공격 전체를 구현하지 않았음을 본문에서 구분했다.
- problems/8_블록암호.zip. 변형 7라운드 구조, 라운드 키 permutation, 50,000쌍과 중간 상태 누출의 원문 자료다.
- solutions/solve_08_aes_key.py. 누출 completion, key-byte constraint join, exhaustive full-key 생성과 전체 검증을 재현하는 구현이다.